A SMALL THEME FOR MODERN SLIDES

Odoo Core Development

Install, modules, fields, ORM, and environments

Weblearns Academy

Odoo Full-Stack Developer Program

Senior Developer Lecture Materials

July 2026

ROADMAP

Agenda

1. Install, CLI, and configuration
2. Modules, manifests, and addons paths
3. Field design and relational models
4. ORM overrides and recordset APIs

TOC

METHOD

How this deck is taught

PATTERN

Objectives → teaching → examples → pitfalls → lab

LIVE CODING

Type commands with learners; freeze a reference commit

SAFETY

Use disposable DBs; never demo sudo shortcuts in production

Odoo 17 | ORM | Fields | CLI

1 O01 – O05

Odoo Core Development · teaching block 1

O01 · OBJECTIVES

Install Odoo 17 on Ubuntu 22.04

- Prepare Ubuntu 22.04 for Odoo 17 development.
- Install system packages, Python dependencies, Node tooling, and wkhtmltopdf.
- Connect the installation to a PostgreSQL role.

> Lab target — Install Odoo 17 from source on Ubuntu 22.04, activate the virtual environment, and run odoo-bin --version.

O01 · TEACHING 1

Install Odoo 17 on Ubuntu 22.04

An Odoo installation is a Python application, a PostgreSQL database client, a set of system libraries, and a front-end asset toolchain. Treat the server setup as part of the application, because missing libraries often appear later as PDF, image, or asset errors.

Ubuntu 22.04 is a common base for Odoo 17 because it provides a stable Python and package ecosystem. Even in training, use a documented setup so students can reproduce bugs and compare results.

Install Odoo 17 on Ubuntu 22.04 (continued)

A virtual environment keeps Odoo's Python packages separate from the operating system. This matters when multiple Odoo versions or experiments live on the same machine.

PostgreSQL should already have a dedicated role for Odoo. The role usually needs CREATEDB in development, while production permissions should be reviewed more strictly.

wkhtmltopdf remains important for many Odoo PDF reports. Use a known compatible package and test report rendering early instead of discovering the issue during invoice printing.

O01 · TEACHING 3

Install Odoo 17 on Ubuntu 22.04 (continued)

After installation, start with a small database and install only base modules needed for the lab. A minimal first boot is easier to debug than a heavily customized environment.

O01 · KEY POINTS

What to remember

- Document every system package required by the Odoo instance.
- Use a Python virtual environment for source-based installs.
- Prepare PostgreSQL before first Odoo startup.
- Test PDF rendering as part of installation validation.

O01 · EXAMPLE 1

Source install outline

Source install outline

This prepares a source checkout suitable for development and keeps Python dependencies inside the local virtual environment.

```
sudo apt update
sudo apt install -y git python3.10-venv python3-dev build-essential     libxml2-dev libxslt1-dev libldap2-dev libsasl2-dev lib

git clone --depth 1 --branch 17.0 https://github.com/odoo/odoo.git /opt/odoo/odoo17
cd /opt/odoo/odoo17
python3 -m venv venv
source venv/bin/activate
pip install --upgrade pip wheel
pip install -r requirements.txt
```

Common mistakes

- Installing Python packages globally and later mixing Odoo versions.
- Skipping wkhtmltopdf validation until the first customer invoice must be printed.
- Using the postgres superuser as the Odoo database user.
- Starting from too many addons before the base installation is proven.

> Caution — Validate fixes in a disposable database before production.

O01 · EXTRA

Install checklist

- System packages installed
- Python virtual environment active
- requirements.txt installed
- PostgreSQL role can connect
- wkhtmltopdf tested

O02 · OBJECTIVES

Start odoo-bin for the First Time

- Run Odoo from the source tree with explicit options.
- Understand addons paths, database host options, and startup logs.
- Use --stop-after-init for controlled initialization.

> Lab target — Start Odoo once with explicit CLI options and once with a config file, then compare the first 30 log lines.

O02 · TEACHING 1

Start odoo-bin for the First Time

odoo-bin is the main entry point for source-based Odoo. Running it manually is the best way to learn what the server is loading before hiding it behind systemd or an IDE.

The addons path determines which modules Odoo can discover. A wrong path can make a valid custom module invisible, while duplicate module names across paths can create confusing behavior.

Database options tell Odoo how to reach PostgreSQL. In local development, host 127.0.0.1 with a dedicated user is explicit and avoids peer authentication surprises.

Start odoo-bin for the First Time (continued)

Startup logs show configuration, addons discovery, database connections, HTTP routes, and module loading. Senior developers read the first failure in the log, not only the final stack trace.

--stop-after-init is useful for installing or upgrading modules from the CLI and then exiting. It gives deterministic command-line workflows for CI, migrations, and training labs.

A first run should use a fresh database and a small module set. Once the server can start cleanly, add custom addons and debugging tools incrementally.

O02 · KEY POINTS

What to remember

- odoo-bin is the source checkout entry point.
- Addons path order affects module discovery.
- --stop-after-init is essential for scripted installs and upgrades.
- Read startup logs from the first error forward.

O02 · EXAMPLE 1

Create and start a training database

Create and start a training database

The first command initializes the database and exits; the second starts the server using a configuration file.

```
./odoo-bin     --addons-path=addons     --db_host=127.0.0.1     --db_user=odoo17     --db_password=odoo17     --database=odoo1

./odoo-bin -c /etc/odoo17.conf --database=odoo17_training
```

O02 · PITFALLS

Common mistakes

- Letting Odoo guess database settings and then debugging the wrong authentication path.
- Putting a custom addons path before core addons without understanding name collisions.
- Ignoring warnings during startup because the web UI appears to load.
- Using --init repeatedly when --update is the intended operation.

> Caution — Validate fixes in a disposable database before production.

O03 · OBJECTIVES

Odoo CLI Commands Overview

- Recognize the most useful odoo-bin command-line options.
- Install, update, test, scaffold, and shell from the CLI.
- Choose command options appropriate for development and maintenance.

> Lab target — Run --help, scaffold a throwaway module, and write the exact command that would update it in a named database.

Odoo CLI Commands Overview

The Odoo CLI is more than a way to start the HTTP server. It is also how developers install modules, run upgrades, create scaffolds, execute tests, open shells, and control database selection.

Common options such as -c, -d, -i, -u, --addons-path, and --stop-after-init are the daily vocabulary of Odoo development. Knowing them makes deployments repeatable instead of dependent on browser clicks.

The shell command opens an Odoo environment in Python. It is powerful for diagnostics, but it can change real data, so use it with the same care as psql.

Odoo CLI Commands Overview (continued)

Test options allow module tests to run without manually clicking through workflows. A serious custom module should have at least a small test surface that can be run from the CLI.

CLI commands should be copied into documentation, deployment scripts, or CI jobs once they are proven. If a release depends on a developer's memory, it is not a release process.

Different Odoo versions add or change options, so always check --help for the installed source tree. Training should teach the pattern, not just one memorized command.

O03 · KEY POINTS

What to remember

- -c selects the configuration file.
- -d selects the database.
- -i installs modules; -u upgrades modules.
- --stop-after-init makes install or upgrade commands exit.
- odoo-bin shell gives direct ORM access.

O03 · EXAMPLE 1

Daily CLI commands

Daily CLI commands

These commands cover the most common lifecycle actions: inspect, install, upgrade, debug, scaffold, and test.

```
./odoo-bin --help
./odoo-bin -c odoo.conf -d odoo17_training -i sale --stop-after-init
./odoo-bin -c odoo.conf -d odoo17_training -u my_module --stop-after-init
./odoo-bin shell -c odoo.conf -d odoo17_training
./odoo-bin scaffold academy_course /opt/odoo/custom_addons
./odoo-bin -c odoo.conf -d odoo17_training --test-enable --stop-after-init
```

O03 · PITFALLS

Common mistakes

- Using -i on a module that is already installed when -u is needed.
- Running shell commands against the wrong database.
- Forgetting --stop-after-init in scripts and leaving a server process running.
- Assuming browser-based module upgrades are equivalent to documented release commands.

> Caution — Validate fixes in a disposable database before production.

O04 · OBJECTIVES

The Odoo Configuration File

– Create an odoo.conf suitable for development.
– Configure database, addons, logging, workers, and admin password options.
– Keep secrets and environment-specific values out of source control.

Lab target — Create a local odoo.conf, start Odoo with -c, and prove from the logs that your custom addons path was loaded.

The Odoo Configuration File

The configuration file turns a long command line into a repeatable server definition. It should make the runtime obvious to another developer who has never used your laptop.

Database settings define where Odoo connects and which credentials it uses. In development they are often explicit, while production may use environment management or secret files around the config.

addons_path is one of the most important settings in the file. It must include core addons, enterprise addons if used, and custom addons in a deliberate order.

The Odoo Configuration File (continued)

admin_passwd protects database management operations such as creating, duplicating, dropping, and backing up databases through the web manager. It is not the same as an Odoo user password.

Worker and limit settings should reflect the deployment mode. Development commonly uses one process with live reload, while production needs worker sizing, memory limits, time limits, and reverse proxy settings.

Never commit production secrets into a training repository or custom module. Provide a sample config and document how real values are supplied on each machine.

O04 · KEY POINTS

What to remember

– Use odoo.conf to make startup repeatable.
– admin_passwd controls database manager actions.
– addons_path must include every module location in the right order.
– Keep production secrets outside versioned code.

O04 · EXAMPLE 1

Development odoo.conf

Development odoo.conf

This sample is explicit enough for training while still showing which values should be changed for real environments.

```
[options]
addons_path = /opt/odoo/odoo17/addons,/opt/odoo/custom_addons
data_dir = /var/lib/odoo17
db_host = 127.0.0.1
db_port = 5432
db_user = odoo17
db_password = odoo17
admin_passwd = change-this-dev-password
log_level = info
logfile = /var/log/odoo/odoo17.log
xmlrpc_port = 8069
limit_time_cpu = 120
limit_time_real = 240
```

Common mistakes

– Committing real db_password or admin_passwd values.
– Forgetting one addons path and wondering why a module does not appear.
– Using production worker settings for local debugging.
– Changing config values but starting Odoo with a different -c file.

> Caution — Validate fixes in a disposable database before production.

O05 · OBJECTIVES

Database Manager URLs and Disabling Database Manager

- Use the Odoo database manager routes safely in development.
- Understand the risk of exposing database management in production.
- Disable database manager access with configuration.

> Lab target — Open the database manager in a lab instance, create a database, then set list_db = False and confirm the list no longer appears.

O05 · TEACHING 1

Database Manager URLs and Disabling Database Manager

Odoo includes browser routes for creating, duplicating, restoring, backing up, and dropping databases. They are convenient in training but must be treated as administrative tools.

The database manager is usually reached through /web/database/manager, with related routes for create, backup, restore, duplicate, and drop. Access is protected by the master password, but exposure still increases risk.

In production, many teams disable list_db and block database manager routes at the reverse proxy. The principle is simple: customers should reach the application, not database administration screens.

O05 · TEACHING 2

Database Manager URLs and Disabling Database Manager (continued)

dbfilter also matters because it controls which database a request can select. A good dbfilter prevents one Odoo service from accidentally offering unrelated databases on the same PostgreSQL server.

The master password must be strong and private if database manager remains enabled. It authorizes destructive actions, so it belongs in secret management rather than a shared wiki.

For training, keep database manager enabled only while teaching the feature. Then show how to disable it so learners understand both convenience and operational safety.

O05 · KEY POINTS

What to remember

– /web/database/manager exposes administrative database actions.
– admin_passwd is the master password for these operations.
– list_db = False hides database listing and is common in production.
– Reverse proxies can block database manager routes.

O05 · EXAMPLE 1

Production-oriented database manager settings

Production-oriented database manager settings

The config hides database listing and the reverse proxy blocks database manager routes before they reach Odoo.

```
[options]
admin_passwd = use-a-secret-manager-value
list_db = False
dbfilter = ^%d$
proxy_mode = True

# Nginx example
location ~* /web/database/ {
    deny all;
}
```

O05 · PITFALLS

Common mistakes

– Leaving database manager visible on a public production domain.
– Using a weak admin_passwd because normal users cannot see it after login.
– Setting dbfilter incorrectly and making valid databases unreachable.
– Assuming list_db = False replaces network and proxy hardening.

> Caution — Validate fixes in a disposable database before production.

2 O06 – O10

Odoo Core Development · teaching block 2

O06 · OBJECTIVES

Database CLI and Upgrade CLI Workflows

- Create, initialize, update, and test databases from the command line.
- Use upgrade commands safely for module development.
- Separate database operations from browser habits.

Lab target — Install a scaffolded module with -i, change its manifest or view, then update it with -u and --stop-after-init.

O06 · TEACHING 1

Database CLI and Upgrade CLI Workflows

Database work in Odoo should be scriptable. A command that creates a database, installs modules, and exits is easier to review than a sequence of browser clicks.

The -i option installs modules into a database and should be used for first installation. The -u option updates already installed modules and is the normal path after changing Python fields, views, security, or data files.

--stop-after-init turns Odoo into a batch command. It loads the registry, performs install or update work, runs enabled tests if requested, and exits with a status code.

Database CLI and Upgrade CLI Workflows (continued)

For module upgrades, keep the database selected explicitly with -d. Accidentally upgrading a customer copy, staging database, or wrong training database is a common and preventable mistake.

When tests are part of the command, failures should stop the release. Do not hide test failures behind a server that keeps running and makes the deployment look successful.

A good upgrade habit includes taking a backup, stopping extra workers or cron where needed, running the update command, reviewing logs, and doing a focused smoke test.

O06 · KEY POINTS

What to remember

- -i is for installation; -u is for updates.
- Always specify the target database.
- --stop-after-init makes commands automation-friendly.
- Upgrade workflows should include backup and smoke testing.

O06 · EXAMPLE 1

Repeatable module upgrade

Repeatable module upgrade

The first command updates a module, and the second adds test execution for a release-quality upgrade check.

```
DB=odoo17_training
CONF=/etc/odoo17.conf

./odoo-bin -c "$CONF" -d "$DB" -u academy_course --stop-after-init

./odoo-bin -c "$CONF" -d "$DB"     -u academy_course     --test-enable     --stop-after-init     --log-level=test
```

O06 · PITFALLS

Common mistakes

- Running -u all casually on a large or customized production database.
- Not backing up before structural module upgrades.
- Forgetting that Odoo shell and upgrade commands can write real data.
- Using browser upgrades as the only deployment procedure.

> Caution — Validate fixes in a disposable database before production.

O07 · OBJECTIVES

PyCharm Configuration for Odoo

- Configure a Python interpreter and run configuration for Odoo.
- Pass Odoo CLI parameters from the IDE.
- Debug server startup and custom module code.

> Lab target — Create a PyCharm run configuration that starts your training database and stops at a breakpoint in a custom model method.

PyCharm Configuration for Odoo

An IDE configuration should mirror the command line that already works. If PyCharm starts Odoo differently than the terminal, debugging becomes a second environment problem.

Point PyCharm at the virtual environment used for the Odoo checkout. This gives code completion for installed dependencies and prevents accidental imports from the system Python.

The script path is usually odoo-bin, and parameters should include -c and the target database or development flags. Working directory should be the Odoo source root unless your project layout intentionally differs.

PyCharm Configuration for Odoo (continued)

Breakpoints work best when workers are disabled and code reload behavior is understood. In development, start with a single process before adding worker-based production settings.

Mark custom addons and the Odoo source as project roots when useful for navigation. Good indexing makes model inheritance, XML ids, and controller code easier to inspect.

Keep IDE-only configuration out of deployment assumptions. The IDE helps developers move faster, but the terminal command remains the source of truth for automation.

O07 · KEY POINTS

What to remember

– Use the same virtual environment in terminal and PyCharm.
– Set odoo-bin as the script path.
– Pass -c and -d explicitly in run parameters.
– Disable workers for straightforward breakpoint debugging.

O07 · EXAMPLE 1

PyCharm run configuration values

PyCharm run configuration values

These settings make the IDE run the same Odoo server that the terminal command would run.

```
Script path:
  /opt/odoo/odoo17/odoo-bin

Parameters:
  -c /etc/odoo17.conf -d odoo17_training --dev=xml,qweb

Working directory:
  /opt/odoo/odoo17

Environment:
  PYTHONUNBUFFERED=1
```

Common mistakes

- Using a different interpreter in PyCharm than the one used to install requirements.txt.
- Debugging with workers enabled and wondering why breakpoints are skipped.
- Leaving the database name empty and attaching to the wrong database.
- Changing PyCharm parameters without updating the documented CLI command.

> Caution — Validate fixes in a disposable database before production.

O08 · OBJECTIVES

Scaffold a Module and Use Multiple Addons Paths

- Generate a module scaffold with odoo-bin.
- Understand custom module directory structure.
- Configure multiple addons paths safely.

> Lab target — Scaffold academy_course, add its parent directory to addons_path, update the app list, and install the module from the CLI.

O08 · TEACHING 1

Scaffold a Module and Use Multiple Addons Paths

Scaffolding creates a conventional starting point for a custom module. It is not magic, but it saves time and teaches the files Odoo expects.

A custom module is discovered because its directory sits under one of the configured addons paths and contains a manifest file. The directory name is the technical module name used in CLI commands.

Multiple addons paths are normal in real projects. A typical setup includes community addons, enterprise addons, first-party custom addons, and sometimes vendor addons.

O08 · TEACHING 2

Scaffold a Module and Use Multiple Addons Paths (continued)

Path order matters when modules share a technical name. Avoid duplicate names entirely because they make upgrades and debugging unpredictable.

The scaffold contains models, views, security, controllers, demo, and tests placeholders depending on the template. Remove or fill placeholders so the module is intentional, not a pile of unused files.

A clean addons layout makes onboarding easier. Developers should know where core code ends, where vendor code lives, and where the team's custom ownership begins.

O08 · KEY POINTS

What to remember

- A module is a directory with a manifest.
- The directory name is the technical module name.
- addons_path can contain several comma-separated roots.
- Duplicate module names across paths are dangerous.

O08 · EXAMPLE 1

Scaffold and configure paths

Scaffold and configure paths

The scaffold command creates the module directory, and the config makes Odoo discover it.

```
mkdir -p /opt/odoo/custom_addons
./odoo-bin scaffold academy_course /opt/odoo/custom_addons

# odoo.conf
addons_path = /opt/odoo/odoo17/addons,/opt/odoo/enterprise,/opt/odoo/custom_addons

./odoo-bin -c /etc/odoo17.conf -d odoo17_training     -i academy_course --stop-after-init
```

Common mistakes

- Putting the module directory itself in addons_path instead of its parent directory.
- Using spaces around paths that are later copied incorrectly into service files.
- Keeping duplicate technical module names in different paths.
- Installing a scaffold before reviewing its manifest, security, and views.

> Caution — Validate fixes in a disposable database before production.

O09 · OBJECTIVES

Manifest Files, Dependencies, and Auto Install Modules

- Write a correct __manifest__.py for Odoo 17.
- Declare dependencies and data files deliberately.
- Use auto_install only for glue modules that should install automatically.

> Lab target — Create a manifest for a training module that depends on mail, includes security before views, and installs cleanly from the CLI.

O09 · TEACHING 1

Manifest Files, Dependencies, and Auto Install Modules

The manifest is the module contract Odoo reads before loading code and data. It tells Odoo the module's name, version, dependencies, category, data files, assets, license, and install behavior.

Dependencies control load order and registry availability. If your model inherits sale.order, the sale module belongs in depends; relying on a user's installed apps is fragile.

Data file order matters because XML records can reference records loaded earlier in the same module or in dependencies. Security files typically load before views that expose the model.

Manifest Files, Dependencies, and Auto Install Modules (continued)

auto_install is for modules that connect other modules when their dependencies are present. It is not a shortcut to force every database to install a feature.

The manifest should be boring and accurate. Inflated dependencies slow installations, while missing dependencies create intermittent failures depending on what happens to be installed.

Version values help release and migration discipline. Even if Odoo does not enforce semantic versioning for custom modules, your team should use versions to communicate change scope.

O09 · KEY POINTS

What to remember

– depends declares modules required before this module loads.
– data file order is load order.
– auto_install is for glue modules.
– Keep manifests accurate and minimal.

O09 · EXAMPLE 1

Odoo 17 manifest

Odoo 17 manifest

The manifest declares precise dependencies, ordered data files, and normal install behavior for an application module.

```
{
    "name": "Academy Course Management",
    "version": "17.0.1.0.0",
    "category": "Education",
    "summary": "Manage training courses and sessions",
    "depends": ["base", "mail", "product"],
    "data": [
        "security/ir.model.access.csv",
        "views/academy_course_views.xml",
        "views/academy_session_views.xml",
        "views/menu.xml",
    ],
    "demo": ["demo/academy_demo.xml"],
    "application": True,
# ... truncated for slide
```

Common mistakes

- Forgetting a dependency because the module happens to be installed in the developer database.
- Loading views before security access rules.
- Setting auto_install to True on a normal business application module.
- Leaving installable False after copying a scaffold.

> Caution — Validate fixes in a disposable database before production.

O10 · OBJECTIVES

Custom Module Structure

– Organize Python, XML, security, data, demo, and test files.

– Import models correctly through __init__.py files.

– Keep module ownership clear as the codebase grows.

Lab target — Create the directories for academy_course, add import files, and confirm a simple model appears in Settings > Technical > Models.

O10 · TEACHING 1

Custom Module Structure

A module structure is a map for both Odoo and future developers. Consistent directories reduce cognitive load when a bug report mentions a model, view, security rule, or scheduled action.

Python packages must be imported for model classes to register. The top-level __init__.py imports models, and models/__init__.py imports the individual Python files.

Security files define who can access the model and under what record rules. Without access rights, a beautiful view can still fail with an access error as soon as a normal user opens it.

Custom Module Structure (continued)

Views describe the user interface and should reference model fields that already exist at module load time. XML ids become stable references used by menus, actions, tests, and other modules.

Data and demo files serve different purposes. Data is normally required for the module to work, while demo should be safe sample content for training and development databases.

Tests belong close to the module because they describe expected module behavior. Even a small test for create, access, and a computed field is valuable during upgrades.

O10 · KEY POINTS

What to remember

- __init__.py imports are required for model registration.
- security/ir.model.access.csv is usually loaded before views.
- data is functional; demo is sample content.
- Tests should travel with the module.

O10 · EXAMPLE 1

Module layout

Module layout

The layout separates responsibilities and shows the import chain Odoo needs to register models.

```
academy_course/
  __init__.py
  __manifest__.py
  models/
    __init__.py
    academy_course.py
    academy_session.py
  security/
    ir.model.access.csv
    academy_security.xml
  views/
    academy_course_views.xml
    menu.xml
  data/
# ... truncated for slide
```

O10 · PITFALLS

Common mistakes

– Creating a Python model file but forgetting to import it.
– Putting required records in demo data and then missing them in production.
– Letting one large models.py file grow without domain boundaries.
– Adding views without corresponding access rights for target users.

> Caution — Validate fixes in a disposable database before production.

3 011 – 015

Odoo Core Development · teaching block 3

O11 · OBJECTIVES

_name, _table, and Model Identity

- Define a new Odoo model with _name.
- Understand default table naming and when to use _table.
- Choose technical names that support long-term maintenance.

Lab target — Create academy.course, install the module, and inspect the generated academy_course table in PostgreSQL.

O11 · TEACHING 1

_name, _table, and Model Identity

_name is the model's technical identifier in the Odoo registry. It is used in Python, XML actions, security files, domains, external APIs, and logs.

By default, Odoo turns dots in _name into underscores for the PostgreSQL table name. A model named academy.course normally stores rows in academy_course.

_table can override the physical table name. Use it rarely, usually for legacy integration, SQL views, or situations where a stable existing table name must be preserved.

O11 · TEACHING 2

_name, _table, and Model Identity (continued)

Technical names should be unique and module-scoped. A good name such as academy.course is clearer than course.course and less likely to collide with another addon.

Changing _name after data exists is a migration, not a rename in one file. Security XML ids, views, relations, ir.model metadata, and existing tables may all need migration work.

Use _description to make technical screens and logs friendlier. Small metadata choices help administrators understand custom models years later.

O11 · KEY POINTS

What to remember

- _name identifies the model in the registry.
- Default table name replaces dots with underscores.
- _table overrides the physical table name.
- Changing model identity after release requires migration.

O11 · EXAMPLE 1

Model identity

Model identity

The class defines a new registry model and uses an explicit table name that matches Odoo's default convention.

```
from odoo import fields, models


class AcademyCourse(models.Model):
    _name = "academy.course"
    _description = "Academy Course"
    _table = "academy_course"

    name = fields.Char(required=True)
    code = fields.Char(required=True, index=True)
    active = fields.Boolean(default=True)
```

O11 · PITFALLS

Common mistakes

- Renaming _name in a released module without a migration plan.
- Using generic model names that collide with vendor or future modules.
- Setting _table unnecessarily and hiding the default naming convention.
- Forgetting _description and making technical screens harder to read.

> Caution — Validate fixes in a disposable database before production.

O12 · OBJECTIVES

Field Overview and Simple Field Options

- Recognize common field declaration options.
- Use required, readonly, index, help, default, copy, and tracking deliberately.
- Understand how fields connect Python models to database columns and views.

> Lab target — Add five fields with different options to academy.course and confirm their metadata with fields_get in Odoo shell.

O12 · TEACHING 1

Field Overview and Simple Field Options

Odoo fields describe both storage and business semantics. A field declaration can create a database column, expose metadata to views, enable search behavior, and participate in tracking or computed logic.

required enforces input at the ORM and user interface level, while SQL constraints may still be needed for deep invariants. index improves common search patterns but adds database write cost.

readonly affects how the field is edited through views and ORM conventions, but it is not a security boundary by itself. Access rights and record rules remain the real permission controls.

Field Overview and Simple Field Options (continued)

default values can be constants, callables, or context-driven helpers. Use defaults to reduce user work, but avoid hiding business decisions that should be explicit.

copy controls whether a field is duplicated when a record is copied. This is important for sequence numbers, external references, and state fields that should not carry over.

tracking integrates with mail.thread to log changes in the chatter. It is valuable for business audit trails, but excessive tracking can create noise and mail_message growth.

O12 · KEY POINTS

What to remember

– Field declarations carry storage, UI, and ORM metadata.
– Indexes should match real searches.
– readonly is not a security mechanism.
– copy=False is important for unique references and workflow state.

O12 · EXAMPLE 1

Common field options

Common field options

The fields demonstrate common options that affect validation, indexing, duplication, help text, defaults, and chatter tracking.

```
from odoo import fields, models


class AcademyCourse(models.Model):
    _name = "academy.course"
    _inherit = ["mail.thread"]
    _description = "Academy Course"

    name = fields.Char(required=True, tracking=True)
    code = fields.Char(required=True, index=True, copy=False)
    active = fields.Boolean(default=True)
    notes = fields.Text(help="Internal teaching notes.")
    state = fields.Selection(
        [("draft", "Draft"), ("published", "Published")],
# ... truncated for slide
```

O12 · PITFALLS

Common mistakes

- Adding index=True to fields that are rarely searched.
- Expecting readonly=True to stop writes from server-side code.
- Letting copy=True duplicate external identifiers.
- Tracking every field and making chatter unusable.

> Caution — Validate fixes in a disposable database before production.

O13 · OBJECTIVES

Text-Oriented Fields: Char, Text, and Html

- Choose between Char, Text, and Html fields.
- Use size, translate, sanitize, and help options appropriately.
- Render text fields safely in views.

> Lab target — Add Char, Text, and Html fields to a model, place them in a form view, and compare how each appears in the UI.

O13 · TEACHING 1

Text-Oriented Fields: Char, Text, and Html

Char is for short values such as names, codes, references, phone numbers, and labels. It communicates that the content is concise and often searchable.

Text is for longer plain text such as internal notes, descriptions, and instructions. It does not imply rich formatting, which keeps content simpler for exports and integrations.

Html stores rich text and is common for website content, email bodies, terms, and formatted descriptions. Because it can contain markup, sanitization and trust boundaries matter.

Text-Oriented Fields: Char, Text, and Html (continued)

translate=True is useful for user-facing labels or descriptions that should differ by language. Do not mark operational codes or external references as translatable.

The widget chosen in XML affects the editing experience. A Text field can use a textarea-like widget, while Html commonly uses Odoo's rich text editor.

Search behavior differs in practice because short indexed fields are easier to optimize than large bodies of text. Use the field that matches the business meaning rather than storing everything as Html.

O13 · KEY POINTS

What to remember

– Char is for short structured text.
– Text is for long plain text.
– Html is for rich formatted content and needs sanitization awareness.
– translate=True belongs on user-facing language content.

O13 · EXAMPLE 1

Course text fields and XML

Course text fields and XML

The model separates searchable codes, plain descriptions, and rich syllabus content instead of using one text type for everything.

```
class AcademyCourse(models.Model):
    _name = "academy.course"
    _description = "Academy Course"

    name = fields.Char(required=True, translate=True)
    code = fields.Char(required=True, size=32, index=True)
    description = fields.Text(translate=True)
    syllabus_html = fields.Html(string="Syllabus", sanitize=True, translate=True)

<!-- views/academy_course_views.xml -->
<field name="description" placeholder="Plain text course summary"/>
<field name="syllabus_html" widget="html"/>
```

O13 · PITFALLS

Common mistakes

– Using Html for plain operational notes.
– Making external codes translatable.
– Expecting a Char size to replace a real business validation.
– Rendering user-provided HTML without considering sanitization.

> Caution — Validate fixes in a disposable database before production.

O14 · OBJECTIVES

Boolean, Selection, Integer, Float, and Json Fields

- Model flags, states, counts, quantities, and flexible payloads.
- Choose numeric fields carefully for business meaning.
- Use Json fields without replacing relational design.

> Lab target — Create a state field with three values, add a button method that changes the state, and store one sample integration payload in a Json field.

O14 · TEACHING 1

Boolean, Selection, Integer, Float, and Json Fields

Boolean fields represent yes-or-no state and are often used for toggles such as active, published, billable, or requires approval. They are simple, but too many booleans can hide a workflow that should be a Selection.

Selection fields represent a controlled set of values. They are ideal for workflow state when the allowed values are known and transitions can be managed in methods.

Integer fields are appropriate for counts and whole-number settings. Float fields are appropriate for approximate decimal quantities, but financial amounts usually need Monetary fields.

Boolean, Selection, Integer, Float, and Json Fields (continued)

Float precision in Odoo is normally controlled through decimal precision settings or digits. Do not assume Python floating point behavior is acceptable for accounting decisions.

Json fields store structured Python dictionaries and lists as JSON data. They are useful for integration payloads, configuration fragments, or analytics data that does not deserve first-class relational fields yet.

Json should not become a dumping ground for data that users search, secure, relate, or report on every day. Once a JSON key becomes core business data, promote it to a real field.

O14 · KEY POINTS

What to remember

– Use Selection for controlled workflow states.
– Use Integer for whole counts and Float for non-money quantities.
– Use Monetary for currency amounts instead of bare Float.
– Json is flexible but weaker than relational fields for core data.

O14 · EXAMPLE 1

Operational scalar fields

Operational scalar fields

The fields show a workflow state, whole-number capacity, decimal duration, and optional integration metadata.

```
class AcademySession(models.Model):
    _name = "academy.session"
    _description = "Academy Session"

    active = fields.Boolean(default=True)
    state = fields.Selection(
        [("draft", "Draft"), ("confirmed", "Confirmed"), ("done", "Done")],
        default="draft",
        required=True,
    )
    seat_count = fields.Integer(default=12)
    duration_hours = fields.Float(digits=(16, 2))
    integration_payload = fields.Json(copy=False)
```

O14 · PITFALLS

Common mistakes

– Using several booleans where one Selection state would be clearer.
– Using Float for currency amounts that must reconcile.
– Storing searchable operational data only inside Json.
– Changing Selection keys without migrating existing database values.

> Caution — Validate fixes in a disposable database before production.

O15 · OBJECTIVES

Date and Datetime Fields

- Use Date and Datetime fields correctly in Odoo 17.
- Understand UTC storage and user timezone conversion.
- Write safe defaults and domains for date ranges.

> Lab target — Add Date and Datetime fields, create records from two user timezones, and compare the UI value to the raw PostgreSQL value.

O15 · TEACHING 1

Date and Datetime Fields

Date fields store calendar dates without time of day. They are appropriate for birthdays, accounting dates, deadlines, validity dates, and business days.

Datetime fields store a timestamp and Odoo treats them as UTC internally. The web client displays them in the user's timezone, which is helpful but can surprise developers inspecting raw PostgreSQL values.

Use fields.Date.today and fields.Datetime.now for defaults rather than evaluating Python date functions at import time. Defaults must run when the record is created, not when the server starts.

Date and Datetime Fields (continued)

Date range domains should be explicit about inclusive and exclusive boundaries. For Datetime fields, a half-open range is often safer than trying to include the last second of a day.

Context-aware date helpers are useful when user timezone matters for business interpretation. A scheduled action running at midnight UTC may not match a user's local business day.

Avoid storing date strings in Char fields. Real Date and Datetime fields give better validation, widgets, searches, grouping, and reporting.

O15 · KEY POINTS

What to remember

- Date has no time; Datetime has timestamp semantics.
- Odoo stores datetimes in UTC.
- Use callable defaults such as fields.Date.today.
- Use half-open ranges for Datetime domains.

O15 · EXAMPLE 1

Date fields and safe domains

Date fields and safe domains

The example uses callable defaults and a half-open monthly range that avoids boundary confusion.

```
class AcademySession(models.Model):
    _name = "academy.session"

    start_date = fields.Date(default=fields.Date.today, required=True)
    start_at = fields.Datetime(default=fields.Datetime.now, required=True)
    deadline = fields.Date()

def sessions_this_month(self):
    start = fields.Date.start_of(fields.Date.today(), "month")
    end = fields.Date.add(start, months=1)
    return self.search([
        ("start_date", ">=", start),
        ("start_date", "<", end),
    ])
```

O15 · PITFALLS

Common mistakes

– Writing default=datetime.now() with parentheses in the field declaration.
– Comparing raw UTC database values to a user's local expectation without conversion.
– Using Char fields for dates.
– Using <= end_of_day strings for Datetime ranges instead of a clear next boundary.

> Caution — Validate fixes in a disposable database before production.

4 O16 – O20

Odoo Core Development · teaching block 4

O16 · OBJECTIVES

Relational Fields: Many2one, One2many, and Many2many

- Model core relationships with Odoo relational fields.
- Understand inverse fields and relation tables.
- Use ondelete, domain, context, and check_company carefully.

> Lab target — Create course, session, and tag models, then inspect the generated foreign key column and Many2many relation table.

O16 · TEACHING 1

Relational Fields: Many2one, One2many, and Many2many

Relational fields are the heart of Odoo's data model. Most business workflows are built by connecting documents, partners, companies, products, users, and accounting records through relationships.

Many2one stores a foreign key to one related record. It is the most common relation and often appears in database tables as a column ending with _id.

One2many is the inverse view of a Many2one. It does not store a separate list in the parent table; it shows child records whose inverse field points back to the parent.

Relational Fields: Many2one, One2many, and Many2many (continued)

Many2many stores pairs of ids in a relation table. It is useful for tags, attendees, allowed users, and other cases where both sides can have many records.

ondelete controls what should happen when a referenced record is removed. In business systems, restrict or set null is often safer than cascade unless child data is purely dependent.

Domains and contexts on relational fields improve user experience by filtering choices and defaulting values. They should support business rules, not replace server-side validation where integrity matters.

O16 · KEY POINTS

What to remember

– Many2one stores the foreign key.
– One2many depends on an inverse Many2one.
– Many2many uses a relation table.
– Choose ondelete behavior based on business data ownership.

O16 · EXAMPLE 1

Course relationships

Course relationships

The parent course sees sessions through One2many, while each session stores the real course_id foreign key.

```
class AcademyCourse(models.Model):
    _name = "academy.course"

    name = fields.Char(required=True)
    session_ids = fields.One2many("academy.session", "course_id")
    tag_ids = fields.Many2many("academy.tag", string="Tags")


class AcademySession(models.Model):
    _name = "academy.session"

    name = fields.Char(required=True)
    course_id = fields.Many2one(
        "academy.course",
# ... truncated for slide
```

O16 · PITFALLS

Common mistakes

- Defining One2many without a valid inverse Many2one field.
- Using cascade deletion on records that have audit or accounting meaning.
- Expecting a Many2many to preserve ordering like line items.
- Using UI domains as the only protection for business integrity.

> Caution — Validate fixes in a disposable database before production.

O17 · OBJECTIVES

Reference and Related Fields

- Use Reference fields for controlled polymorphic links.
- Use Related fields to expose values from linked records.
- Decide when stored related fields are worth the cost.

> Lab target — Add a related course field on session activity, make one version stored and one non-stored, then compare fields_get metadata.

Reference and Related Fields

A Reference field can point to records from different models. It is useful for generic links, but it is weaker than a normal Many2one because the database cannot enforce a single foreign key.

Use Reference only when polymorphism is truly part of the business requirement. Many designs become simpler and safer with explicit Many2one fields to the expected model.

Related fields expose a value through a relationship path, such as session.course_id.company_id. They reduce duplication and keep the UI close to the data users need.

Reference and Related Fields (continued)

A non-stored related field is computed on access and does not create a column. A stored related field is searchable and groupable more efficiently, but it must be recomputed when dependencies change.

Related fields can be readonly by default because their source is another record. If users should edit the source value, make the relationship and ownership clear in the view.

For reporting and domains, store related fields only when there is a real search or performance reason. Storing every convenience value adds recomputation work and schema noise.

O17 · KEY POINTS

What to remember

– Reference fields are polymorphic but less constrained.
– Related fields expose values through relation paths.
– store=True improves searchability at recomputation cost.
– Prefer explicit Many2one when only one target model is valid.

O17 · EXAMPLE 1

Reference and related fields

Reference and related fields

The stored course_id can support search or grouping, while the instructor remains a lightweight display convenience.

```
class AcademyActivity(models.Model):
    _name = "academy.activity"

    name = fields.Char(required=True)
    resource_ref = fields.Reference(
        selection=[("academy.course", "Course"), ("academy.session", "Session")],
        string="Related Resource",
    )
    session_id = fields.Many2one("academy.session")
    course_id = fields.Many2one(related="session_id.course_id", store=True)
    instructor_id = fields.Many2one(related="session_id.instructor_id", store=False)
```

Common mistakes

- Using Reference fields when a normal Many2one would enforce the relationship.
- Storing related fields without a search or reporting reason.
- Expecting database foreign keys from Reference values.
- Making related fields editable without explaining where the value is actually stored.

> Caution — Validate fixes in a disposable database before production.

O18 · OBJECTIVES

Compute Fields

- Write computed fields with correct dependencies.
- Choose between stored and non-stored compute fields.
- Use inverse and search methods when the business case requires them.

> Lab target — Add session_count to academy.course, update sessions, and confirm the count changes after module upgrade.

O18 · TEACHING 1

Compute Fields

Computed fields derive their value from other fields or records. They keep business logic in Python instead of forcing users or integrations to maintain redundant values manually.

@api.depends tells Odoo which changes should trigger recomputation. Missing dependencies create stale values, while overly broad dependencies waste time during writes.

Non-stored computed fields are calculated when read. Stored computed fields create database columns and are recomputed when dependencies change, which makes them searchable and groupable in more cases.

Compute Fields (continued)

Every record in self must receive a value in the compute method. Compute methods that accidentally assign only some records create cache errors that can be difficult for beginners to understand.

Inverse methods allow a computed field to be edited by translating the entered value back to source fields. Use them only when that reverse mapping is clear and stable.

Search methods let non-stored computed fields participate in domains. They should return an equivalent domain, not perform arbitrary recordset filtering in Python for large datasets.

O18 · KEY POINTS

What to remember

– Use @api.depends accurately.
– Assign a value for every record in the compute method.
– store=True trades recomputation cost for search and grouping benefits.
– Inverse and search methods are advanced tools for specific cases.

O18 · EXAMPLE 1

Stored computed field

Stored computed field

The compute method assigns every course a count and stores the result for efficient display and search.

```
from odoo import api, fields, models


class AcademyCourse(models.Model):
    _name = "academy.course"

    session_ids = fields.One2many("academy.session", "course_id")
    session_count = fields.Integer(compute="_compute_session_count", store=True)

    @api.depends("session_ids")
    def _compute_session_count(self):
        for course in self:
            course.session_count = len(course.session_ids)
```

O18 · PITFALLS

Common mistakes

– Forgetting @api.depends and wondering why values are stale.
– Assigning a computed value only inside an if branch.
– Setting store=True on expensive fields without considering write impact.
– Using compute fields to hide data that should be normalized into real relational fields.

> Caution — Validate fixes in a disposable database before production.

O19 · OBJECTIVES

Monetary, Binary, and Image Fields

- Represent currency amounts with Monetary fields.
- Store files and images with Binary and Image fields.
- Understand attachment storage and UI widgets.

> Lab target — Add a fee, currency, brochure, and image to academy.course, then upload a file and confirm an ir.attachment was created.

Monetary, Binary, and Image Fields

Monetary fields pair an amount with a currency field. This is the correct pattern for prices, fees, budgets, and totals that users expect to format by currency.

A Monetary field normally references a res.currency record through currency_field. Without a clear currency, an amount is just a number and can be misread in multi-company environments.

Binary fields store file-like data and often use attachment=True so data is stored as an ir.attachment rather than directly in the model table. This keeps large payloads out of frequently scanned business tables.

Monetary, Binary, and Image Fields (continued)

Image fields build on binary storage with image-specific resizing and widgets. They are appropriate for logos, profile pictures, product images, and course thumbnails.

Files and images affect backup strategy because Odoo attachments commonly live in the filestore. A database dump without the filestore can leave Binary and Image fields broken.

Access to attachments must match business expectations. Do not assume that because a file is stored indirectly it is automatically safe for every user to retrieve.

O19 · KEY POINTS

What to remember

- Use Monetary with a currency field for money.
- Binary fields often store data as attachments.
- Image fields support image-specific UI and processing.
- Database and filestore backups must stay together.

O19 · EXAMPLE 1

Money and files

Money and files

The example links amounts to company currency and stores course files through Odoo's attachment mechanism.

```
class AcademyCourse(models.Model):
    _name = "academy.course"

    currency_id = fields.Many2one(
        "res.currency",
        default=lambda self: self.env.company.currency_id,
        required=True,
    )
    fee = fields.Monetary(currency_field="currency_id")
    brochure = fields.Binary(attachment=True)
    brochure_filename = fields.Char()
    image_1920 = fields.Image(max_width=1920, max_height=1920)

<!-- XML -->
# ... truncated for slide
```

O19 · PITFALLS

Common mistakes

– Using Float for prices instead of Monetary.
– Forgetting the currency field in multi-company data.
– Backing up the database but not the filestore.
– Storing large files directly on heavily used models without considering attachment behavior.

> Caution — Validate fixes in a disposable database before production.

O20 · OBJECTIVES

Default ORM Methods and Recordsets

- Understand recordsets and default model method style.
- Use browse, create, write, copy, and unlink safely.
- Recognize when methods operate on multiple records.

> Lab target — In Odoo shell, create three courses, batch update them with write, duplicate one, and check exists on a deleted id.

O20 · TEACHING 1

Default ORM Methods and Recordsets

Odoo's ORM represents records as recordsets, even when there is only one record. This design lets methods work consistently on zero, one, or many records.

Model methods receive self as a recordset or model environment depending on how they are called. A method should be clear about whether it supports multiple records or requires exactly one.

browse creates a recordset from ids without immediately verifying that every id exists. exists is often used later when the code must discard deleted or inaccessible records.

O20 · TEACHING 2

Default ORM Methods and Recordsets (continued)

create, write, copy, and unlink are core lifecycle methods. They are powerful extension points but also high-risk because many modules may override the same methods in the inheritance chain.

The ORM handles access rules, field conversions, computed fields, invalidation, and relational commands. Direct SQL skips these behaviors and should be reserved for carefully controlled cases.

A senior Odoo developer thinks in recordsets, not loops over raw ids. This leads to clearer code and fewer unnecessary database queries.

O20 · KEY POINTS

What to remember

– self is usually a recordset.
– browse does not guarantee records still exist.
– Core lifecycle methods should be extended carefully.
– Prefer ORM operations unless direct SQL is deliberately justified.

O20 · EXAMPLE 1

Recordset basics in shell

Recordset basics in shell

The shell example shows recordset search, batch write, existence checking, copy, and create through the ORM.

```
courses = env["academy.course"].search([("active", "=", True)], limit=10)
courses.write({"state": "published"})

course = env["academy.course"].browse(42)
if course.exists():
    duplicate = course.copy({"name": "%s (Copy)" % course.name})

env["academy.course"].create({
    "name": "Odoo Developer Bootcamp",
    "code": "ODOO-BOOT",
})
```

Common mistakes

- Assuming self always contains exactly one record.
- Looping over ids and performing one search per id.
- Using direct SQL for changes that need access rules or recomputation.
- Calling browse and then assuming the record exists in the database.

> Caution — Validate fixes in a disposable database before production.

5 O21 – O25

Odoo Core Development · teaching block 5

O21 · OBJECTIVES

Override create and write

- Override create with @api.model_create_multi.
- Override write without breaking batch behavior.
- Call super and preserve the inheritance chain.

> Lab target — Override create to assign a sequence and write to stamp a publication datetime only when the state first becomes published.

O21 · TEACHING 1

Override create and write

create and write are the most common lifecycle overrides in custom modules. They are also easy to damage because they sit on hot paths used by imports, forms, automated actions, and other modules.

In Odoo 17, create should normally use @api.model_create_multi so one call can create several records efficiently. Code that assumes a single vals dictionary can slow imports or break batch creation.

write receives one vals dictionary applied to every record in self. If behavior depends on old values, capture them before super because after super the recordset reflects the new data.

O21 · TEACHING 2

Override create and write (continued)

Always call super unless you are intentionally replacing core behavior, which is rare. The inheritance chain may include mail tracking, website behavior, accounting rules, and other installed modules.

Validation in create or write should be targeted and explainable. Prefer constraints for invariant rules and use overrides for workflow side effects, sequence assignment, or integration hooks.

Avoid writing to the same records inside write without guards because recursion is easy. If a follow-up update is required, make it explicit and ensure it will not call itself forever.

O21 · KEY POINTS

What to remember

– Use @api.model_create_multi for create overrides.
– write applies vals to every record in self.
– Capture old values before super when needed.
– Call super to preserve installed module behavior.

O21 · EXAMPLE 1

Safe create and write overrides

Safe create and write overrides

The create override supports batch creation, and write captures old state before calling super.

```
from odoo import api, fields, models


class AcademyCourse(models.Model):
    _name = "academy.course"

    name = fields.Char(required=True)
    code = fields.Char(copy=False)
    published_at = fields.Datetime(copy=False)
    state = fields.Selection([("draft", "Draft"), ("published", "Published")], default="draft")

    @api.model_create_multi
    def create(self, vals_list):
        for vals in vals_list:
# ... truncated for slide
```

O21 · PITFALLS

Common mistakes

- Using @api.model and a single vals dict for batch create code.
- Forgetting to return the result of super.
- Reading old values after super and losing the transition information.
- Creating recursive writes without guards.

> Caution — Validate fixes in a disposable database before production.

O22 · OBJECTIVES

Custom Methods, copy, and unlink

- Write business methods that operate on recordsets.
- Override copy to reset duplicated values.
- Override unlink to protect important records.

> Lab target — Add a publish button method, prevent deletion of published records, and verify duplicate records return to draft state.

O22 · TEACHING 1

Custom Methods, copy, and unlink

Custom methods should express business actions in the language of the module. Button handlers such as action_confirm or action_archive are clearer than placing workflow logic directly in views.

Methods called by buttons often require one record, but batch actions may pass many. Use ensure_one only when the method truly cannot operate on multiple records.

copy duplicates a record and lets you provide default values for the duplicate. Reset fields such as state, sequence code, external references, and counters that should not be cloned.

Custom Methods, copy, and unlink (continued)

unlink deletes records and should be restricted when records have business history. In many Odoo workflows, archiving with active=False is safer than permanent deletion.

Raise UserError or ValidationError with messages that help the user fix the problem. A stack trace or generic access error is not a good business experience.

Keep custom methods small enough to test. If an action performs validation, state change, notification, and external API calls, split helper methods around clear responsibilities.

O22 · KEY POINTS

What to remember

– Custom methods should express business actions.
– Use ensure_one only for truly single-record actions.
– Override copy to reset duplicated business identifiers.
– Prefer archive over unlink for historical business records.

O22 · EXAMPLE 1

Business methods and deletion protection

Business methods and deletion protection

The methods implement a publish action, safe duplication defaults, and deletion protection for published courses.

```
from odoo import _, models
from odoo.exceptions import UserError


class AcademyCourse(models.Model):
    _name = "academy.course"

    def action_publish(self):
        for course in self:
            if not course.session_ids:
                raise UserError(_("Add at least one session before publishing."))
        self.write({"state": "published"})

    def copy(self, default=None):
# ... truncated for slide
```

Common mistakes

- Calling ensure_one at the top of every method by habit.
- Letting copy duplicate unique business references.
- Deleting records that should remain for audit history.
- Raising technical exceptions that do not guide the user.

> Caution — Validate fixes in a disposable database before production.

O23 · OBJECTIVES

exists, search, and Domain Operators

- Use exists to clean recordsets.
- Write search domains with common operators.
- Combine domain logic with &, |, and !.

> Lab target — Write three domains: exact state, OR condition, and inactive-inclusive search using with_context(active_test=False).

O23 · TEACHING 1

exists, search, and Domain Operators

search is the main ORM method for finding records. It accepts a domain, optional offset, limit, order, and count behavior through related methods.

A domain is a list representation of logical conditions. Each tuple contains a field, an operator, and a value, while prefix operators such as | and ! combine expressions.

Common operators include =, !=, in, not in, like, ilike, child_of, >=, <=, and =?. The right operator makes code more readable and can change whether indexes are useful.

O23 · TEACHING 2

exists, search, and Domain Operators (continued)

exists returns the subset of a recordset that still exists and is accessible. It is useful after browse on external ids, user input, or records that may have been deleted by another transaction.

Domains are translated to SQL with ORM context, access rights, active_test, and company behavior in mind. A domain is not just a string filter; it is part of Odoo's security-aware data access layer.

Complex domains should be named or built in helper methods. A long inline domain copied across models is a maintenance risk and often hides subtle logic differences.

O23 · KEY POINTS

What to remember

– search takes a domain and returns a recordset.
– Domain operators describe SQL-like comparisons and Odoo hierarchy logic.
– Prefix | means OR, & means AND, and ! means NOT.
– exists filters out missing records from a recordset.

O23 · EXAMPLE 1

Search domains

Search domains

The examples show normal AND behavior, explicit OR behavior, ordering, limiting, and existence cleanup.

```
Course = self.env["academy.course"]

published = Course.search([
    ("active", "=", True),
    ("state", "=", "published"),
    ("name", "ilike", "odoo"),
], order="name", limit=20)

draft_or_expensive = Course.search([
    "|",
    ("state", "=", "draft"),
    ("fee", ">=", 1000),
])

# ... truncated for slide
```

Common mistakes

- Building domains with string concatenation instead of domain lists.
- Forgetting that active_test may hide inactive records.
- Using ilike for every search even when exact equality is intended.
- Browsing arbitrary ids and assuming all records exist.

> Caution — Validate fixes in a disposable database before production.

O24 · OBJECTIVES

search_count, read, read_group, and search_read

- Choose the right read method for lists, counts, and aggregates.
- Use read_group for grouped reporting.
- Avoid loading unnecessary fields or records.

> Lab target — Build a small dashboard method that returns draft count, the ten latest published courses, and fees grouped by state.

O24 · TEACHING 1

search_count, read, read_group, and search_read

Not every data question needs full recordsets with every field prefetched. Odoo provides specialized methods for counts, raw reads, combined search-and-read, and grouped aggregation.

search_count returns the number of records matching a domain without fetching all records. It is ideal for badges, warnings, and decisions that only need a count.

read returns dictionaries for fields on an existing recordset. It is useful for RPC-like output or integration code that needs simple structures rather than record methods.

search_count, read, read_group, and search_read (continued)

search_read combines search and read in one call and is common in controllers and API-style endpoints. It should still use fields, limit, and domain deliberately.

read_group performs grouped aggregation similar to SQL GROUP BY while respecting ORM access and context. It is useful for dashboards and summary widgets.

Performance depends on shape. Fetching 10 fields from 80 records is different from fetching 80 fields from 10,000 records, so be precise about fields and limits.

O24 · KEY POINTS

What to remember

– Use search_count for counts.
– Use read for dictionaries from known records.
– Use search_read for search plus field extraction.
– Use read_group for grouped aggregates.

O24 · EXAMPLE 1

Reading and grouping

Reading and grouping

The methods answer count, list, and grouped reporting questions without loading more data than needed.

```
Course = self.env["academy.course"]

draft_count = Course.search_count([("state", "=", "draft")])

rows = Course.search_read(
    [("state", "=", "published")],
    fields=["name", "code", "fee"],
    limit=10,
    order="name",
)

summary = Course.read_group(
    domain=[("active", "=", True)],
    fields=["fee:sum", "id:count"],
# ... truncated for slide
```

O24 · PITFALLS

Common mistakes

- Using len(search(domain)) instead of search_count for large datasets.
- Calling search_read without a limit in controller code.
- Expecting read_group output to look exactly like SQL rows.
- Fetching all fields when only names and ids are required.

> Caution — Validate fixes in a disposable database before production.

O25 · OBJECTIVES

name_create, default_get, _name_search, and get_view

- Customize quick creation and default values.
- Improve Many2one search behavior with _name_search.
- Understand when get_view customization is appropriate.

> Lab target — Make academy.course searchable by code in a Many2one field and add a default company through default_get.

O25 · TEACHING 1

name_create, default_get, _name_search, and get_view

name_create supports quick creation from relational widgets. It should create a valid minimal record, not bypass required business setup that the user must provide.

default_get computes default values for new records based on fields, context, company, user, and business rules. It is a good place for defaults that need more logic than a field declaration.

_name_search controls how records are found when users type in Many2one fields. It should return ids matching the user's text across meaningful fields such as code and name.

name_create, default_get, _name_search, and get_view (continued)

Search customization must respect access rules and performance. A convenient _name_search that scans large text fields can slow every form that uses the model.

get_view can modify view architecture before it reaches the client. It is powerful but should be used sparingly because dynamic XML manipulation is harder to reason about than normal view inheritance.

Prefer declarative XML view inheritance and context defaults first. Use these hooks when the behavior is model-level and cannot be expressed cleanly in standard configuration.

O25 · KEY POINTS

What to remember

- name_create handles quick creation from relational fields.
- default_get returns context-aware defaults.
- _name_search improves lookup behavior.
- get_view is powerful and should be rare.

O25 · EXAMPLE 1

Lookup and defaults hooks

Lookup and defaults hooks

The model supports quick creation, company-aware defaults, and lookup by either course name or code.

```
from odoo import api, models


class AcademyCourse(models.Model):
    _name = "academy.course"
    _rec_name = "name"

    @api.model
    def name_create(self, name):
        record = self.create({"name": name, "code": name.upper().replace(" ", "-")[:32]})
        return record.name_get()[0]

    @api.model
    def default_get(self, fields_list):
# ... truncated for slide
```

Common mistakes

- Using name_create to create incomplete records that later fail business rules.
- Overriding default_get and ignoring the requested fields_list.
- Writing _name_search domains that ignore args passed by the caller.
- Using get_view for changes that normal XML inheritance can handle.

> Caution — Validate fixes in a disposable database before production.

6 O26 – O30

Odoo Core Development · teaching block 6

O26 · OBJECTIVES

ensure_one, filtered, mapped, sorted, grouped, fields_get, and get_metadata

- Use recordset helper methods idiomatically.
- Inspect model metadata from the ORM.
- Avoid unnecessary loops and single-record assumptions.

> Lab target — Use Odoo shell to filter published courses, map their names, group by state, and print fields_get metadata for three fields.

O26 · TEACHING 1

ensure_one, filtered, mapped, sorted, grouped, fields_get, and get_metadata

Recordset helpers make Odoo code concise and expressive. They operate on recordsets and usually preserve ORM behavior better than manually building lists of ids.

ensure_one asserts that a method is working with exactly one record. It is appropriate for single-record actions and computed display helpers, but it should not be used to avoid thinking about batch behavior.

O26 · TEACHING 2

ensure_one, filtered, mapped, sorted, grouped, fields_get, and get_metadata (continued)

filtered keeps records matching a predicate or field expression, mapped extracts fields or calls, sorted orders records, and grouped partitions records by a key. These methods make business code read closer to the business question.

fields_get returns field metadata such as type, string, required, readonly, selection, and help. It is useful for dynamic UIs, debugging, and training exercises about what Odoo knows about a model.

O26 · TEACHING 3

ensure_one, filtered, mapped, sorted, grouped, fields_get, and get_metadata (continued)

get_metadata returns technical metadata such as XML ids and creation or update information. It helps when diagnosing data loaded from XML or deciding whether a record is safe to reference externally.

Use helpers on reasonably sized recordsets. For very large datasets, push filtering and grouping into search domains or read_group so PostgreSQL does the heavy work.

O26 · KEY POINTS

What to remember

– ensure_one is an assertion, not a default habit.
– filtered, mapped, sorted, and grouped express recordset transformations.
– fields_get exposes field metadata.
– get_metadata helps trace XML ids and record provenance.

O26 · EXAMPLE 1

Recordset helpers

Recordset helpers

The code shows common helpers and a correct single-record action that uses ensure_one intentionally.

```
courses = self.env["academy.course"].search([])

published = courses.filtered(lambda course: course.state == "published")
names = published.mapped("name")
ordered = published.sorted(key=lambda course: course.fee, reverse=True)
by_state = courses.grouped("state")

field_info = self.env["academy.course"].fields_get(["name", "state", "fee"])
metadata = published[:1].get_metadata()

def action_open_form(self):
    self.ensure_one()
    return {
        "type": "ir.actions.act_window",
# ... truncated for slide
```

Common mistakes

– Calling ensure_one in methods that could naturally support batch actions.
– Using filtered on thousands of records when a search domain would be better.
– Assuming mapped always returns a recordset; scalar fields return Python lists.
– Ignoring metadata tools and hardcoding guesses about fields or XML ids.

> Caution — Validate fixes in a disposable database before production.

O27 · OBJECTIVES

search_fetch and Efficient Data Loading

- Use search_fetch when records and selected fields are needed together.
- Understand prefetching and field loading behavior.
- Reduce query chatter in read-heavy custom code.

> Lab target — Write one method with search and one with search_fetch for the same field access pattern, then compare SQL logs in a small benchmark.

O27 · TEACHING 1

search_fetch and Efficient Data Loading

Odoo's ORM uses caching and prefetching to reduce repetitive database access. Most normal code benefits automatically, but high-volume loops can still create avoidable query chatter.

search_fetch searches for records and fetches specified fields in one deliberate operation. It is useful when the next code path will read a known set of fields from every returned record.

The method keeps recordset semantics while making field-loading intent explicit. This can be cleaner than search followed by manual reads when the code still wants record methods.

search_fetch and Efficient Data Loading (continued)

Do not use search_fetch everywhere by habit. If code only needs ids, search may be enough; if it needs dictionaries for an API response, search_read may be clearer.

Performance work should be measured with logs, tests, or query counters when possible. Odoo caches can make a change look faster in a warm shell while offering little benefit in a real request.

Efficient loading is especially valuable in computed fields, scheduled actions, exports, and controllers. These paths often touch many records and can amplify small ORM mistakes.

O27 · KEY POINTS

What to remember

– search_fetch combines search with deliberate field fetching.
– Use it when you need a recordset and known fields immediately.
– search_read is better when dictionaries are the desired output.
– Measure performance changes before making broad refactors.

O27 · EXAMPLE 1

Fetch fields for a batch job

Fetch fields for a batch job

The batch job declares the fields it will use before looping over the recordset.

```
def _send_course_digest(self):
    courses = self.env["academy.course"].search_fetch(
        [("state", "=", "published")],
        ["name", "code", "fee", "currency_id"],
        limit=500,
        order="name",
    )
    for course in courses:
        body = "%s [%s] costs %s" % (course.name, course.code, course.fee)
        self.env["mail.mail"].create({
            "subject": "Course digest",
            "body_html": body,
            "email_to": "training@example.com",
        })
```

Common mistakes

- Using search_fetch when only a count is needed.
- Fetching many fields just in case.
- Optimizing without measuring query count or runtime.
- Replacing clear search_read API code with recordset code that callers do not need.

> Caution — Validate fixes in a disposable database before production.

O28 · OBJECTIVES

Special Relational Commands 0 through 6

- Use Odoo's relational command tuples correctly.
- Apply commands to One2many and Many2many fields.
- Prefer Command helpers for readable Odoo 17 code.

> Lab target — Create a course with two sessions using Command.create, add two tags with Command.set, then remove one tag with Command.unlink.

O28 · TEACHING 1

Special Relational Commands 0 through 6

Relational commands describe changes to One2many and Many2many fields in create and write calls. They let one ORM call create, update, link, unlink, clear, or replace related records.

The classic tuple commands are numbered 0 through 6. They are compact but easy to misread, so Odoo's Command helper is preferred in modern code for clarity.

Command 0 creates a new related record, command 1 updates an existing related record, and command 2 deletes the related record. For One2many, these are common when editing line items.

Special Relational Commands 0 through 6 (continued)

Command 3 unlinks a relation without deleting the target record, command 4 links an existing record, command 5 clears all links, and command 6 replaces all links with a given id list. Many2many fields use 3, 4, 5, and 6 frequently.

Be careful with command 2 because it deletes the related record, not just the relationship. On business documents, that distinction can affect audit history and accounting integrity.

Relational commands should be built from validated data. Do not accept arbitrary command lists from untrusted external clients without checking allowed operations and target records.

O28 · KEY POINTS

What to remember

– 0 creates, 1 updates, 2 deletes.
– 3 unlinks, 4 links, 5 clears, 6 replaces.
– Use odoo.fields.Command for readable code.
– Delete and unlink are different operations.

O28 · EXAMPLE 1

Command helper examples

Command helper examples

The helper names make the intent clearer than raw numbers while still producing the same ORM commands.

```
from odoo.fields import Command

course = self.env["academy.course"].create({
    "name": "Advanced Odoo",
    "session_ids": [
        Command.create({"name": "ORM Deep Dive"}),
        Command.create({"name": "Performance Lab"}),
    ],
    "tag_ids": [
        Command.link(existing_tag_id),
    ],
})

course.write({
# ... truncated for slide
```

Common mistakes

- Using command 2 when command 3 was intended.
- Replacing all Many2many links with command 6 when the code meant to add one link.
- Passing raw external command lists into write without validation.
- Forgetting that One2many requires values compatible with the child model.

> Caution — Validate fixes in a disposable database before production.

O29 · OBJECTIVES

Active Archive Behavior

- Use the active field for archiving records.
- Understand active_test context in searches and relational fields.
- Choose archive or unlink based on business history.

> Lab target — Add active to academy.course, archive one record, prove normal search hides it, and prove active_test=False finds it.

O29 · TEACHING 1

Active Archive Behavior

Many Odoo models use an active Boolean to archive records instead of deleting them. Archived records remain in the database but are hidden from normal searches and views.

The ORM context key active_test controls whether inactive records are filtered automatically. By default, active_test is usually true, so search results may exclude archived records without an explicit domain.

Archiving is safer for records referenced by historical documents. A partner, product, course, or analytic account may no longer be selectable for new work but still needs to appear on old records.

Active Archive Behavior (continued)

Views often provide Archive and Unarchive actions automatically when a model has active. This creates a consistent user experience across custom and standard models.

Search domains that explicitly mention active can interact with active_test in confusing ways. When diagnosing archived data, use with_context(active_test=False) and then filter intentionally.

Do not use active as a substitute for workflow state. Active means visible or usable, while state represents where a record is in a business process.

O29 · KEY POINTS

What to remember

– active=False archives records.
– active_test controls automatic filtering of inactive records.
– Archive preserves history better than unlink.
– active is not a workflow state field.

O29 · EXAMPLE 1

Archive-aware searches

Archive-aware searches

The lookup includes archived records deliberately, while the archive action hides completed courses without deleting them.

```
class AcademyCourse(models.Model):
    _name = "academy.course"

    active = fields.Boolean(default=True)

def find_any_course(self, code):
    return self.with_context(active_test=False).search([("code", "=", code)], limit=1)

def action_archive_old_courses(self):
    old_courses = self.search([
        ("state", "=", "done"),
        ("active", "=", True),
    ])
    old_courses.write({"active": False})
```

Common mistakes

- Deleting historical records instead of archiving them.
- Forgetting active_test=False when searching for archived records.
- Using active to represent draft, confirmed, and done workflow states.
- Adding custom archive logic when the standard active behavior is enough.

> Caution — Validate fixes in a disposable database before production.

O30 · OBJECTIVES

Environment self.env and Modifying Environment

- Use self.env to access models, user, company, context, and cursor.
- Modify environment with sudo, with_context, with_user, with_company, and with_env.
- Understand the security impact of environment changes.

> Lab target — In Odoo shell, print env.user, env.company, and env.context, then compare search results with normal access, sudo, and active_test=False.

O30 · TEACHING 1

Environment self.env and Modifying Environment

self.env is the gateway to the Odoo runtime. It carries the database cursor, current user, context, registry, company information, cache, and model accessors.

self.env['model.name'] returns a model recordset bound to the same environment. This is how custom code reaches other models without importing their Python classes directly.

with_context creates a new recordset with modified context. Context can drive defaults, language, timezone, active_test, company behavior, and many model-specific flags.

O30 · TEACHING 2

Environment self.env and Modifying Environment (continued)

sudo changes access rights by using superuser mode or a specific user. It is necessary for some technical operations, but it can bypass record rules and expose or modify data the current user should not touch.

with_user and with_company are more explicit tools when behavior should be evaluated as another user or company. They are useful in tests, multi-company logic, and controlled automation.

O30 · TEACHING 3

Environment self.env and Modifying Environment (continued)

Environment changes return new recordsets; they do not mutate every existing variable. A senior developer keeps the elevated or modified environment as narrow as possible and returns to normal permissions quickly.

O30 · KEY POINTS

What to remember

– self.env gives access to models, cursor, user, company, and context.
– with_context changes contextual behavior for a recordset.
– sudo bypasses normal access checks and must be narrow.
– with_user and with_company make user and company changes explicit.

O30 · EXAMPLE 1

Environment patterns

Environment patterns

The method shows context changes, narrow sudo usage, company-scoped work, and model access through self.env.

```
def action_prepare_digest(self):
    Mail = self.env["mail.mail"]
    lang = self.env.user.lang or "en_US"

    courses = self.with_context(active_test=False, lang=lang).search([
        ("state", "=", "published"),
    ])

    admin_course_count = self.sudo().search_count([])

    company_courses = self.with_company(self.env.company).search([
        ("company_id", "=", self.env.company.id),
    ])

# ... truncated for slide
```

Common mistakes

- Using sudo everywhere instead of fixing access rights or record rules.
- Forgetting that with_context returns a new recordset.
- Using self.env.cr for direct SQL when ORM methods would be safer.
- Mixing companies in code without explicit with_company or company domains.

> Caution — Validate fixes in a disposable database before production.

O30 · EXTRA

Security habit

Security habit — Keep sudo blocks as small as possible and never use them to hide missing business security design.

WRAP-UP

You should now be able to...

- O01 Install Odoo 17 on Ubuntu 22.04
- O02 Start odoo-bin for the First Time
- O03 Odoo CLI Commands Overview
- O04 The Odoo Configuration File
- O05 Database Manager URLs and Disabling Database Manager
- O06 Database CLI and Upgrade CLI Workflows
- … and 24 more lessons in this deck

> Remember — Complete outstanding labs before starting the next section.

Odoo 17 | ORM | Fields | CLI

NEXT STEP

Complete the section labs

Open the next presentation deck

Section 04 · Odoo Core

July 2026